1. a)

$$\phi_M(u) = \begin{cases} u^2 & , |u| \le M \\ 2M|u| - M^2 & , |u| > M \end{cases}$$

$$\phi'_M(u) = \begin{cases} 2u & , |u| \le M \\ -2M & , u < -M \\ 2M & , u > M \end{cases}$$

$$\Phi_M(\beta) = \frac{1}{n} \sum_{i=1}^{n} \phi_M(y_i - \beta^T x_i)$$

$$\nabla \Phi_M(\beta) = \frac{1}{n} \sum_{i=1}^{n} \phi'_M(y_i - \beta^T x_i) \cdot (-x_i)$$

$$\nabla \Phi_M(\beta) = -\frac{1}{n} \sum_{i=1}^{n} \begin{cases} 2(y_i - \beta^T x_i) \cdot x_i, & |y_i - \beta^T x_i| \le M \\ -2M \cdot x_i & , y_i - \beta^T x_i < -M \\ 2M \cdot x_i & , y_i - \beta^T x_i > M \end{cases}$$

457_a2

2026-01-28

Qustion 1 b)

```r
library(CVXR)
```

```
##
## Attaching package: 'CVXR'

## The following object is masked from 'package:stats':
##
##     power
```

```r
# Load the data
data <- read.csv("dat_Q1A2.csv")
y <- as.matrix(data[, 1])
X <- as.matrix(data[, 2:4])
n <- nrow(X)
p <- ncol(X)

# Define the Gradient Function
huber_gradient <- function(beta, X, y, M) {
  residuals <- y - X %*% beta
  grad_elements <- matrix(0, nrow = n, ncol = p)

  for (i in 1:n) {
    res <- residuals[i]
    if (abs(res) <= M) {
      grad_elements[i, ] <- -2 * res * X[i, ]
    } else {
      grad_elements[i, ] <- -2 * M * sign(res) * X[i, ]
    }
  }

  # Return the mean gradient across all observations
  return(colMeans(grad_elements))
}

# Gradient Descent Solver
solve_huber_gd <- function(X, y, M, step_size = 0.01, max_iter = 50000, tol = 1e-5) {
  beta <- rep(0, p)

  for (i in 1:max_iter) {
```

```r
    grad <- huber_gradient(beta, X, y, M)
    beta_new <- beta - step_size * grad

    # Check convergence based on loss
    if (sqrt(sum((beta_new - beta)^2)) < tol) {
      return(beta_new)
    }
    beta <- beta_new
  }
  return(beta)
}

# Execution for M values
M_values <- c(0.1, 0.3, 0.5, 0.9)
results <- list()

print("Huber Regression Results (Gradient Descent):")
```

```
## [1] "Huber Regression Results (Gradient Descent):"
```

```r
for (m in M_values) {
  beta_hat <- solve_huber_gd(X, y, m)
  results[[as.character(m)]] <- beta_hat
  cat("\nM = ", m, ":\n")
  print(beta_hat)
}
```

```
##
## M =  0.1 :
## [1]  0.97881888 -0.79464989  0.02894928
##
## M =  0.3 :
## [1]  0.966166134 -0.783549464  0.008508817
##
## M =  0.5 :
## [1]  0.957758794 -0.774368854 -0.005563862
##
## M =  0.9 :
## [1]  0.96863110 -0.79557273 -0.03351702
```

2. a) $f'(x) = 6(x-1)^5 + 4(x-3)^3 + 2(x-5)$

$f''(x) = 30(x-1)^4 + 12(x-3)^2 + 2$

We can see that this function is convex because $f''(x) \geq 0 \ \forall x$.

$(x-1)^4$ and $(x-3)^2$ are guaranteed to be positive terms.

Question 2b)

```r
f_deriv <- function(x) {
  6*(x - 1)^5 + 4*(x - 3)^3 + 2*(x - 5)
}

# Define the second derivative function
f_double_deriv <- function(x) {
  30*(x - 1)^4 + 12*(x - 3)^2 + 2
}

# Newton-Raphson Algorithm
newton_raphson <- function(x0, tol = 1e-12, max_iter = 100) {
  x_curr <- x0

  for (i in 1:max_iter) {
    # Calculate values at current x
    g <- f_deriv(x_curr)
    h <- f_double_deriv(x_curr)

    # Check stopping criterion
    if (abs(g) < tol) {
      cat("Convergence reached in", i-1, "iterations.\n")
      return(x_curr)
    }

    # Update step
    x_curr <- x_curr - g / h
  }

  return(x_curr)
}

# Execute with initial value x0 = 0
x_star <- newton_raphson(x0 = 0)
```

```
## Convergence reached in 7 iterations.
```

```r
deriv_at_x_star <- f_deriv(x_star)

# Report results
print("Minimizer x*: ")
```

```
## [1] "Minimizer x*: "
```

```r
print(x_star)
```

```
## [1] 2.082612
```

```r
print("f'(x*) value: ")
```

```
## [1] "f'(x*) value: "
```

```r
print(deriv_at_x_star)
```

```
## [1] 1.29674e-13
```

3. a) $\sigma(z) = \log(1+e^{-z})$

$$\sigma'(z) = \frac{1}{(1+e^{-z})} \cdot (-e^{-z}) = \frac{-e^{-z}}{1+e^{-z}} \frac{(e^{z})}{(e^{z})} = \frac{-e^{-z+z}}{e^{z}+e^{-z+z}} = \frac{-1}{e^{z}+1}$$

$$\sigma''(z) = 1 \cdot (e^{z}+1)^{-2} \cdot e^{z} = \frac{e^{z}}{(e^{z}+1)^{2}}$$

Again because $\frac{e^{z}}{(e^{z}+1)^{2}}$, e^{z} is a positive function, $(e^{z}+1)^{2}$ is too

$\sigma''(z) > 0 \ \forall \ z \in \mathbb{R}$, so $\sigma(z)$ must be convex.

Let $z = \beta^T x$

Then $f(x) = \sigma(\beta^T x) = \sigma(z)$

$\beta^T x$ is a linear transformation of x,

and we already showed $\sigma''(z) > 0$ for all z, then $\sigma''(\beta^T x)$ must be positive

Thus, $f(x) = \log(1+e^{-\beta^T x})$ must be convex.

b) $Y_i = \beta^*_0 + \beta^*_1 x_{i1} + \dots + \beta^*_p x_{ip} + \varepsilon_i$

$\hat{\sigma}^2 = \frac{1}{n-p-1} \|Y - \hat{Y}\|^2$, $\hat{Y} = \hat{H}Y$, $\hat{H}$ is the projection matrix

$= X(X^T X)^{-1} X^T$

$$\hat{Y} = X\hat{\beta} = \underbrace{X\underbrace{(X^T X)^{-1} X^T Y}_{\hat{\beta}}}_{\hat{H}} = \hat{H}Y$$

$$Y - \hat{Y} = (I - \hat{H})Y = (I - \hat{H})(X\beta^* + \varepsilon)$$

$$= (I - \hat{H})X\beta^* + (I - \hat{H})\varepsilon$$

$\hat{H}X = X$ because columns of X lie in the subspace onto which $\hat{H}$ projects

$$= (X - \hat{H}X)\beta^* + (I - \hat{H})\varepsilon$$

$$= (I - \hat{H})\varepsilon$$

$$\|Y - \hat{Y}\|^2 = ((I - \hat{H})\varepsilon)^T (I - \hat{H})\varepsilon$$

$$= \varepsilon^T (I - \hat{H})^T (I - \hat{H})\varepsilon$$

because $\hat{H}$ is symmetric, idempotent $I - \hat{H}$ is too

$(I - \hat{H})(I - \hat{H}) = I - \hat{H}$

$$= \varepsilon^T (I - \hat{H})\varepsilon$$

$$E[\|Y-\hat{Y}\|^2] = E[\varepsilon^T (I-\hat{H})\varepsilon] = \sum_{i,j} a_{ij}\,\varepsilon_i \varepsilon_j = \sum_i a_{ii}\, E[\varepsilon_i^2] = \sigma^2 \sum_i a_{ii} = \sigma^2\, tr(A)$$

let $A = I - \hat{H}$

$A_{ij} = a_{ij}$

Note: $Rank(H) = p+1$ $\quad Rank(I-H) = n-(p+1)$

thus $tr(I-\hat{H}) = n-p-1.$ $\quad = n-p-1$

$$E[\|Y-\hat{Y}\|^2] = \sigma^2(n-p-1)$$

$$E\left[\frac{1}{n-p-1}\|Y-\hat{Y}\|^2\right] = \sigma^2 \longleftarrow \boxed{\hat{\sigma}^2 = \frac{1}{n-p-1}\|Y-\hat{Y}\|^2}$$

$$\therefore E[\hat{\sigma}^2] = \sigma^2$$

Question 4

4a

```r
train_data <- read.csv("Q4A2_train.csv")
test_data <- read.csv("Q4A2_test.csv")


# Fit the model using all covariates
full_model <- lm(Salary ~ ., data = train_data)

# Predict on test data
pred_all <- predict(full_model, newdata = test_data)

# Calculate Mean Squared Error (MSE)
mse_all <- mean((test_data$Salary - pred_all)^2)

print(mse_all)
```

```
## [1] 0.3986058
```

4b

```r
# Part b
# Perform stepwise selection in 'both' directions starting from the full model
step_model <- step(full_model, direction = "both", trace = 0)

print(summary(step_model))
```

```
##
## Call:
## lm(formula = Salary ~ AtBat + Hits + Walks + Years + CHits +
##     CRuns + CRBI + CWalks + League + Division + PutOuts, data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.09084 -0.38546  0.04807  0.35741  2.56695
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  4.7491837  0.2292716  20.714  < 2e-16 ***
## AtBat       -0.0032120  0.0014240  -2.256  0.02558 *
## Hits         0.0123600  0.0046602   2.652  0.00888 **
## Walks        0.0098601  0.0039155   2.518  0.01287 *
## Years        0.0695315  0.0297560   2.337  0.02081 *
## CHits       -0.0008753  0.0006282  -1.393  0.16561
## CRuns        0.0030976  0.0013379   2.315  0.02199 *
## CRBI         0.0007054  0.0004751   1.485  0.13973
## CWalks      -0.0021773  0.0008054  -2.703  0.00768 **
## LeagueN      0.2147371  0.1065820   2.015  0.04577 *
```

```
## DivisionW   -0.2150362  0.1042048  -2.064  0.04083 *
## PutOuts      0.0004825  0.0002179   2.215  0.02834 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6219 on 146 degrees of freedom
## Multiple R-squared:  0.5314, Adjusted R-squared:  0.4961
## F-statistic: 15.05 on 11 and 146 DF,  p-value: < 2.2e-16
```

```r
print("The selected")
```

```
## [1] "The selected"
```

4c

```r
# Part c
# Predict on test data using the stepwise-selected model
pred_step <- predict(step_model, newdata = test_data)

# Calculate MSE for the selected model
mse_step <- mean((test_data$Salary - pred_step)^2)
print(mse_step)
```

```
## [1] 0.3939498
```

Question 5

Part a

```r
library(glmnet)
```

```
## Loading required package: Matrix

## Loaded glmnet 4.1-10
```

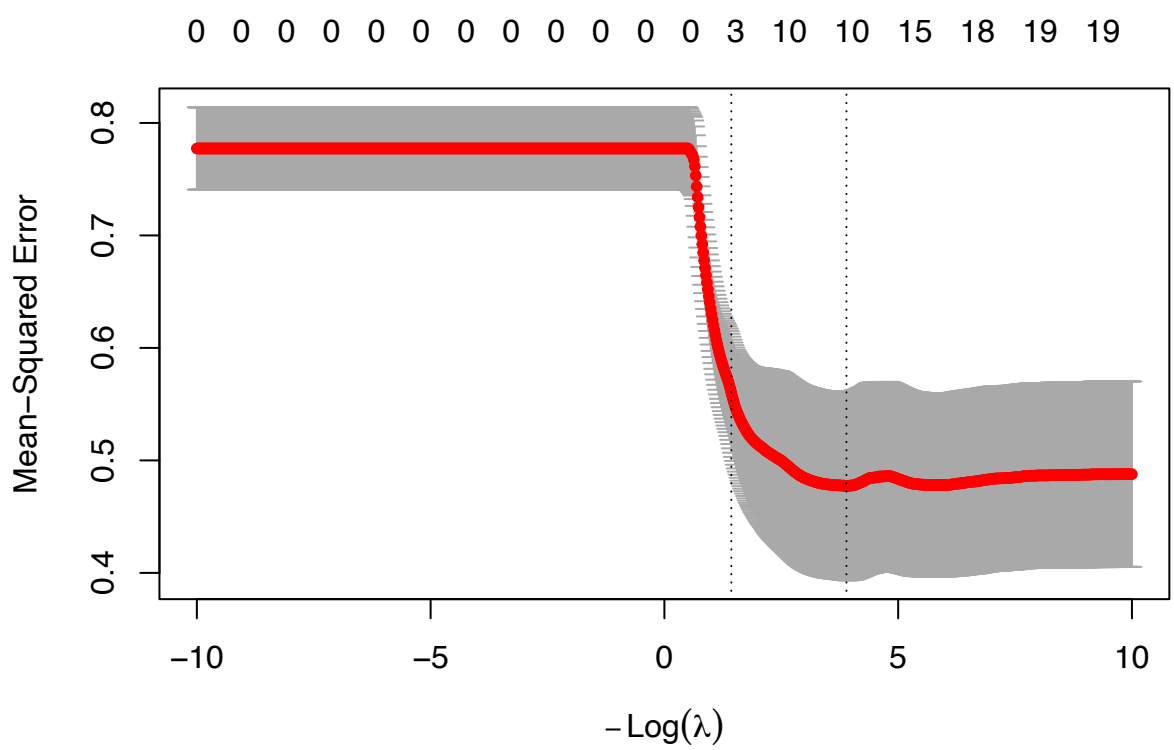
```r
x_train <- model.matrix(Salary ~ ., data = train_data)[, -1]
y_train <- train_data$Salary

x_test <- model.matrix(Salary ~ ., data = test_data)[, -1]
y_test <- test_data$Salary

# Define the requested lambda sequence
lambda_seq <- exp(seq(-10, 10, length = 1000))

# Perform 10-fold CV for Lasso (alpha = 1)
set.seed(11)
cv_lasso <- cv.glmnet(x_train, y_train, alpha = 1, lambda = lambda_seq, nfolds = 10)

plot(cv_lasso)
```



Part b

```r
lasso_1se_coeffs <- coef(cv_lasso, s = "lambda.1se")
print(lasso_1se_coeffs)
```

```
## 20 x 1 sparse Matrix of class "dgCMatrix"
##               lambda.1se
## (Intercept) 5.6202641568
## AtBat       .
## Hits        0.0004094230
## HmRun       .
## Runs        .
## RBI         .
## Walks       .
## Years       .
## CAtBat      .
## CHits       .
## CHmRun      .
## CRuns       0.0005522514
## CRBI        0.0003376558
## CWalks      .
## LeagueN     .
## DivisionW   .
## PutOuts     .
## Assists     .
## Errors      .
## NewLeagueN  .
```

Part c

```r
# Identify the best lambda
best_lambda <- cv_lasso$lambda.min

# Predict on test data using lambda.min
preds <- predict(cv_lasso, s = best_lambda, newx = x_test)

# Calculate and report Test MSE
test_mse_lasso <- mean((y_test - preds)^2)
print("The test Mean Square Error for the Lasso model is:")
```

```
## [1] "The test Mean Square Error for the Lasso model is:"
```

```r
print(test_mse_lasso)
```

```
## [1] 0.3816532
```